

7. Soluțiile vor fi verificate încrucișat folosind MOSS. Nu este permisă partajarea de cod între studenți. Soluțiile care au un grad de similitudine mai mare de 30% vor fi anulate.

2 din 2

- le obligatorii (soluția este depunctată cu câte 2 puncte pentru fiecare cerință ce nu este maxim se pot pierde 8 puncte
1. Denumirea claselor, funcțiilor, testelor unitare, atributelor și a variabilelor se respecta convenția de nume de tip Java Mix CamelCase;
 2. Pattern-urile și clasa ce conține metoda main() sunt definite în pachete distincte ce au forma *cts.numa.prenume.gNrGrupa.denumire_pattern*, *cts.numa.prenume.gNrGrupa.main* (studenții din anul suplimentar trec "as" în loc de gNrGrupa);
 3. Clasele și metodele sunt implementate respectând principiile KISS, DRY și SOLID (atenție la DIP);
 4. Denumirile de clase, metode, variabile, precum și mesajele afișate la consola trebuie să aibă legătura cu subiectul primit (nu sunt acceptate denumiri generice). Funcțional, metodele vor afișa mesaje la consola care să simuleze acțiunea cerută sau vor implementa prelucrări simple.

Se dezvoltă o aplicație software destinată managementului de proiecte

3p. O aplicație de management de proiect trebuie să notifice mai multe componente atunci când starea unei sarcini se schimbă: modulul de raportare, modulul de notificări, calendarul echipei și istoricul proiectului. Componentele interesate trebuie să poată fi adăugate sau eliminate dinamic, fără modificarea clasei care gestionează sarcina. Pentru implementare se va folosi interfața *AbstractSarcinaMonitorizata* și *AbstractModulProiect*.

1.5p. Să se testeze soluția prin:

- crearea unei sarcini monitorizate;
- atașarea mai multor module;
- schimbarea stării sarcinii;
- eliminarea unui modul și repetarea notificării.

3p. Sistemul trebuie să importe sarcini dintr-o aplicație externă de tip task manager. Aplicația internă folosește sarcini cu titlu, prioritate numerică și termen limită, în timp ce aplicația externă folosește descriere, nivel textual de urgență și dată exprimată într-un alt format. Se cere integrarea sarcinilor externe în fluxul intern de planificare, astfel încât planificatorul existent să le poată trata ca sarcini interne. Pentru implementare se va folosi interfața *AbstractSarcinaInterna*.

1.5p. Să se testeze soluția prin:

- planificarea unei sarcini interne;
- planificarea unei sarcini externe integrate;
- folosirea aceluiași planificator pentru ambele tipuri;
- evidențierea conversiei dintre prioritatea textuală externă și prioritatea numerică internă.

```
public interface AbstractSarcinaMonitorizata {
    void adaugaListener(AbstractModulProiect modul);
    void eliminaListener(AbstractModulProiect modul);
    void notificaModule();
}
```

```
public interface AbstractModulProiect {
    void actualizeaza(String stareNoua);
}
```

```
public interface AbstractSarcinaInterna {
    String obtineTitlu();
    int obtinePrioritate();
    LocalDate obtineTermenLimita();
}
```